25900787

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

MGU-UGP (HONOURS) REGULAR/IMPROVEMENT/REAPPEARANCE

EXAMINATION NOVEMBER 2025

FIRST SEMESTER

Discipline Specific Core Course (DSC) - MG1DSCCSC100, MG1DSCECC101 ART OF COMPUTING AND PROBLEM SOLVING

(2024 ADMISSION ONWARDS)

Duration: 1 Hour 30 Minutes Maximum Marks: 50

Part A: Very Short Answer Questions (Answer all) - (10 × 1 = 10 Marks)

1. If a variable is used before it is initialized, what type of error is likely to occur? [U] / [CO1]

Answer: Runtime error or Logical error (1 mark for correct answer)

2. Identify the problem-solving approach that involves combining small, well-defined modules to create a larger system. [K] / [CO1]

Answer: Modular approach / Bottom-up approach / Structured programming (1 mark)

3. A program is written using binary codes (0s and 1s). Identify the type of language this refers to. [K] / [CO1]

Answer: Machine language / Low-level language (1 mark)

4. How do you format a floating-point number to display two decimal places in C? [U] / [CO2]

Answer: %.2f or printf("%.2f", variable); (1 mark)

5. Identify the operator in C that is used as a short form of the if-else statement. [K] / [CO2]

Answer: Ternary operator / Conditional operator / ?: operator (1 mark)

6. A programmer declares a variable with the identifier '1stValue' and initializes it to 10. What will happen when compiling this code? [U] / [CO2]

Answer: Compilation error / Syntax error (variable names cannot start with a digit) (1 mark)

7. When does an 'else' block will be executed in an 'if-else' statement? [U] / [CO3]

Answer: When the if condition is false / evaluates to 0 (1 mark)

8. When does a 'do-while' loop terminate? [U] / [CO3]

Answer: When the condition becomes false / evaluates to 0 (after executing at least once) (1 mark)

9. How do you skip to the next iteration of a loop? [U] / [CO3]

Answer: continue; statement (1 mark)

10. Explain what happens if the condition in a 'while' loop is always true. [U] / [CO3]

Answer: Infinite loop / The loop never terminates (1 mark)

Part B: Short Answer Questions (Answer any 4 out of 6) - (4 × 5 = 20 Marks)

11. Explain the relationship between bits and bytes and their significance in computing. [K] /

[CO1] Answer: (5 marks total)

• Bit: Basic unit of digital information, can be 0 or 1 (1 mark)

• Byte: Collection of 8 bits (1 mark)

• Relationship: 1 byte = 8 bits (1 mark)

• Significance in computing (2 marks)

- Bits used for binary representation
- Bytes used as fundamental unit for memory storage
- Data storage measured in bytes (KB, MB, GB, etc.)
- Character encoding (ASCII uses 1 byte per character)

12. Discuss the purpose of testing and verification in the software development process. [U] /

[CO1] Answer: (5 marks total)

Purpose of testing (2.5 marks)

• To identify errors/bugs in the program

• To ensure program meets requirements

• To validate functionality

• To check reliability and performance

• Example or types of testing mentioned

Purpose of verification: (2.5 marks)

• To check if software is built correctly

• To ensure adherence to specifications

• To validate design and logic

• Code reviews and walkthroughs

• Prevents errors early in development

13. Discuss the fundamental data types in C with examples. [K] / [CO2]

Answer: (5 marks - 1 mark for each data type with example)

• **int:** For integer values (e.g., int age = 25;) (1 mark)

• **float:** For single-precision floating-point (e.g., float price = 99.99;) (1 mark)

• **double:** For double-precision floating-point (e.g., double pi = 3.14159265359;) (1 mark)

• **char:** For single characters (e.g., char grade = 'A';) (1 mark)

• **void:** Represents absence of type (e.g., void function()) (1 mark)

14. How to call a function in C? Explain with an example. [K] / [CO3]

Answer: (5 marks total)

Explanation of function call: (2 marks)

• Function called by its name followed by parentheses

• Arguments passed inside parentheses if required

• **Example with function definition:** (3 marks)

```
Function definition:
int add(int a, int b)
    { return a + b;
}

Function call in main:
int result = add(5, 3);
```

15. Write a C program to reverse a string. [A] / [CO3]

(Correct syntax: 2 marks, Logic: 3 marks, 5 marks total)

Sample Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100],
    temp; int i, j,
    len;

    printf("Enter a string: ");
    scanf("%s", str);

    len = strlen(str);

    for(i = 0, j = len - 1; i < j; i++, j--) {
        temp = str[i];
        str[i] =
        str[j];
        str[j] = temp;
    }

    printf("Reversed string: %s", str);
     return 0;
}
```

16. Write a program to find sum of elements of a one dimensional array. [A] / CO3]

(Correct syntax: 2 marks, Logic: 3 marks, 5 marks total)

Sample Program

```
#include <stdio.h>

int main() {
    int arr[100], n, i, sum = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter
    elements:\n"); for(i = 0;
```

```
    i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for(i = 0; i < n; i++) {
        sum = sum + arr[i];
    }

    printf("Sum = %d", sum);
    return 0;
}
```

Part C: Essay Questions (Answer any 2 out of 3) - (2 × 10 = 20 Marks)

17. Describe the main functions of compilers, interpreters, and assemblers in programming and their roles in code translation. [U] / [CO1]

Answer: (10 marks total)

Compiler: (3 marks)

• Translates entire high-level language program to machine code at once

• Creates executable file that can run independently

• Faster execution but slower translation

• Examples: GCC for C, Turbo C

• Error reporting: All errors shown after compilation

Interpreter: (3 marks)

• Translates and executes program line by line

• No separate executable file created

• Slower execution but immediate feedback

• Examples: Python, JavaScript interpreters

• Error reporting: Stops at first error encountered

Assembler: (3 marks)

• Translates assembly language to machine code

• Works with low-level language (mnemonics to binary)

• One-to-one mapping between assembly and machine instructions

• Examples: NASM, MASM

• Produces object code

Comparison/Role in code translation: (1 mark)

• All serve as translators between human-readable and machine-executable code

18. Discuss the use of variables and constants in programming. Explain different types of constants with examples. [U] / [CO2]

Answer: (10 marks total)

Variables: (4 marks)

• Definition: Named storage locations that can change during program execution

• Purpose: Store data that can be manipulated

• Must be declared before use

• Example: int age = 25; (can be changed later)

• Scope and lifetime discussion

• Naming rules and conventions (1 mark)

Constants: (2 marks)

• Definition: Fixed values that cannot be changed during execution

• Purpose: Store values that remain constant throughout program

• Improves code readability and maintainability

Types of Constants: (4 marks - 1 mark each type)

1. Integer Constants:

• Whole numbers without decimal point

• Example: 100, -50, 0x2A (hexadecimal), 052 (octal)

2. Floating-point Constants:

• Numbers with decimal point

• Example: 3.14, -0.5, 2.5e3 (scientific notation)

3. Character Constants:

• Single character enclosed in single quotes

• Example: 'A', 'x', '\n' (escape sequence)

4. String Literals:

• Sequence of characters in double quotes

• Example: "Hello World", "C Programming"

Using #define or const keyword for defining constants

Example: #define PI 3.14159 or const int MAX = 100;

19. Explain the syntax and functioning of a 'for' loop in C. Implement a 'for' loop for the following scenario:
[A] / [CO3]

Answer: (10 marks total)

Syntax and Functioning: (4 marks)

```
Syntax:
for (initialization; condition; increment/decrement)
{
    // body of loop
}
```

Explanation:

• Initialization: Executed once at the beginning

• Condition: Checked before each iteration

• Increment/Decrement: Executed after each iteration

• Body: Statements executed when condition is true

• Flow: initialization → condition check → body → increment → condition check...

Program Implementation (finding Armstrong numbers between 100-500): (6 marks)

Sample code

```
#include <stdio.h>
#include <math.h>

int main() {
   int num, sum, temp, digit, count;

   printf("Armstrong numbers between 100 and 500:\n");

   for(num = 100; num <= 500; num++) {
   temp = num;
```

```
        sum = 0;
        count = 0;

        // Count digits
        while(temp !=
        0) {
            count++;
            temp = temp / 10;
        }

        temp = num;

        // Calculate sum of digits raised to power
        of count while(temp != 0) {
            digit = temp % 10;
            sum = sum + pow(digit, count);
            temp = temp / 10;
        }

        // Check if Armstrong
        number if(sum == num) {
            printf("%d ", num);
        }
    }

    return 0;
}
```